Progetto 1: Server HTTP Concorrente

Gestione Prenotazione Aule Studio in C

Massimo Mantineo (541924) Pierluca Tino Castorina (545101)

Università degli Studi di Messina
Dipartimento MIFT - Laurea in Informatica

Anno Accademico 2024/2025

1. Obiettivi del Progetto

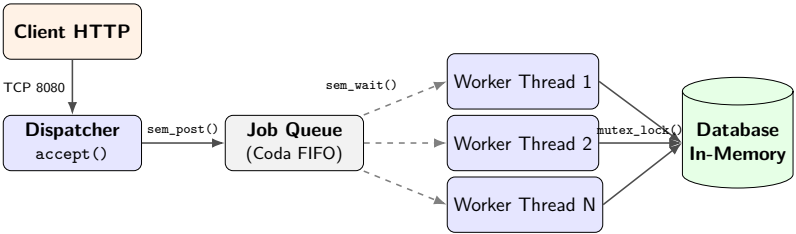
Il Problema

Implementare un server per la gestione di prenotazioni in grado di servire simultaneamente molteplici client, garantendo la coerenza dei dati e riducendo l'overhead computazionale di sistema.

Le Soluzioni Adottate:

- **C Nativo & POSIX:** Utilizzo delle *system call* di basso livello anziché di framework moderni (Node.js, Spring) per massimizzare il controllo delle risorse.
- **Architettura a Thread Pool:** Sostituzione del pattern classico basato su `fork()` con un pool di worker thread pre-allocati.
- **Sincronizzazione Esplicita:** Impiego combinato di Semafori (per la comunicazione di rete) e Mutex (per l'accesso in mutua esclusione al database in memoria).

2. Architettura a Thread Pool



Evidenza: Handshake TCP

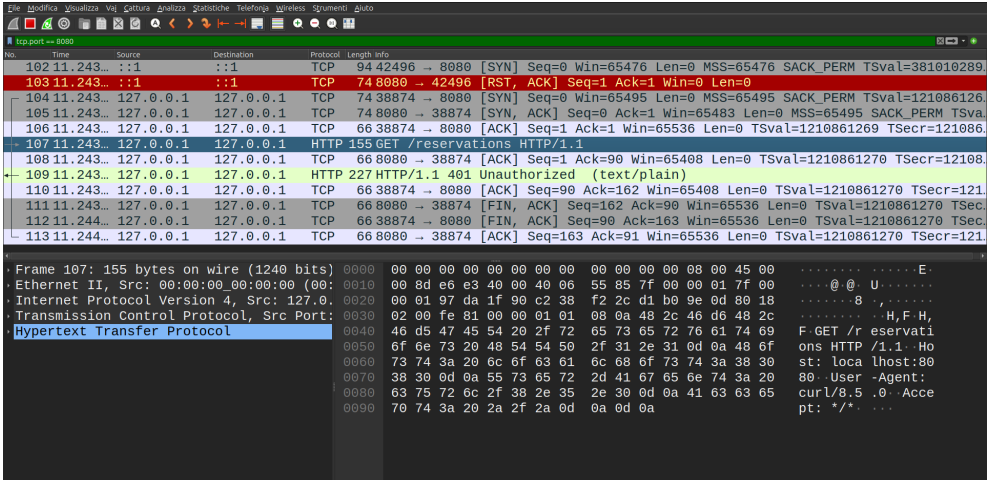


Figura: Tracciato Wireshark del Three-Way Handshake TCP.

Autenticazione

3. Strutture Dati per la Concorrenza

L'architettura poggia su due strutture fondamentali per la gestione dello stato e della sincronizzazione.

```
typedef struct job {
    void (*function)(void*); // Funzione da eseguire
    void* arg;               // Argomento (es. socket descriptor)
    struct job* next;        // Puntatore per coda FIFO
} job_t;

typedef struct {
    pthread_t* threads;        // Array di worker thread
    job_t* queue_head;         // Testa della coda (estrazione)
    job_t* queue_tail;         // Coda della lista (inserimento)
    pthread_mutex_t queue_mutex; // Protezione della coda
    sem_t queue_sem;           // Conteggio task pronti
    int shutdown;              // Flag di terminazione controllata
} thread_pool_t;
```

4. Inizializzazione del Pool

All'avvio, il server alloca le risorse di sincronizzazione e lancia i thread worker.

```
thread_pool_t* thread_pool_init(int num_threads) {
    thread_pool_t* pool = malloc(sizeof(thread_pool_t));

    // Inizializzazione primitiva di sincronizzazione
    pthread_mutex_init(&pool->queue_mutex, NULL);
    sem_init(&pool->queue_sem, 0, 0); // Semaforo parte da 0

    pool->threads = malloc(num_threads * sizeof(pthread_t));
    for (int i = 0; i < num_threads; i++) {
        // Ogni thread esegue la funzione 'thread_worker'
        pthread_create(&pool->threads[i], NULL, thread_worker, (void*)pool);
    }
    return pool;
}
```

5. Il Pattern Produttore-Consumatore

Dispatcher (Produttore)

```
// Accettazione e creazione job
int* client_sock = malloc(sizeof(int));
*client_sock = accept(...);

pthread_mutex_lock(&pool->queue_mutex);
// Aggiunta in coda (FIFO)
pool->queue_tail->next = job;
pool->queue_tail = job;
pthread_mutex_unlock(&pool->queue_mutex);

// Sveglia un thread worker
sem_post(&pool->queue_sem);
```

Worker (Consumatore)

```
while (1) {
    // Attesa passiva (nessun busy-waiting)
    sem_wait(&pool->queue_sem);

    pthread_mutex_lock(&pool->queue_mutex);
    if (pool->shutdown) {
        pthread_mutex_unlock(&pool->
            queue_mutex);
        break; // Shutdown controllato
    }
    // Estrazione job dalla coda...
    pthread_mutex_unlock(&pool->queue_mutex);

    handle_client(job);
}
```

6. Shutdown Sicuro e Gestione Semafori

La chiusura del server richiede un coordinamento per evitare *deadlock* o thread rimasti in attesa sul semaforo.

```
void thread_pool_destroy(thread_pool_t* pool) {
    pthread_mutex_lock(&pool->queue_mutex);
    pool->shutdown = 1; // Segnale di stop
    pthread_mutex_unlock(&pool->queue_mutex);

    // Svegliamo tutti i thread in attesa su sem_wait()
    for (int i = 0; i < pool->num_threads; i++) {
        sem_post(&pool->queue_sem);
    }

    // Join per attendere la terminazione effettiva
    for (int i = 0; i < pool->num_threads; i++) {
        pthread_join(pool->threads[i], NULL);
    }
}
```

7. Thread Safety: Il Database in Memoria

L'accesso alla risorsa condivisa (il vettore delle prenotazioni) è protetto da un Mutex per garantire la coerenza dei dati durante operazioni concorrenti.

```
int rm_create(resource_manager_t* rm, const char* room, ...) {
    pthread_mutex_lock(&rm->mutex); // ACQUISIZIONE LOCK

    if (rm->count >= MAX_RESERVATIONS) {
        pthread_mutex_unlock(&rm->mutex);
        return -1;
    }

    // Scrittura sicura nell'array condiviso
    strncpy(rm->reservations[rm->count].room, room, 127);
    rm->count++;

    pthread_mutex_unlock(&rm->mutex); // RILASCIO LOCK
    return rm->count - 1;
}
```

8. Sincronizzazione ed Eliminazione Atomica

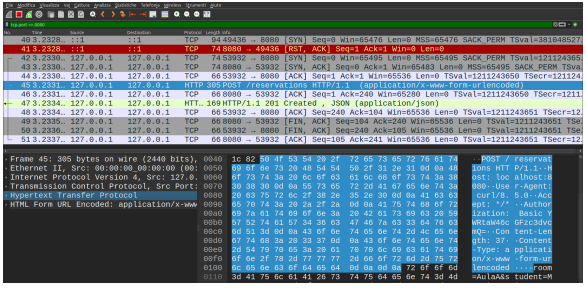
Le operazioni logiche complesse, come la rimozione con ricompattazione (shift), devono avvenire integralmente sotto lock.

```
int rm_delete(resource_manager_t* rm, int id) {
    pthread_mutex_lock(&rm->mutex); // Sezione Critica
    for (int i = 0; i < rm->count; i++) {
        if (rm->reservations[i].id == id) {
            // Shift a sinistra per ricompattare l'array
            for (int j = i; j < rm->count - 1; j++) {
                rm->reservations[j] = rm->reservations[j + 1];
            }
            rm->count--;
            pthread_mutex_unlock(&rm->mutex);
            return 0;
        }
    }
    pthread_mutex_unlock(&rm->mutex);
    return -1;
}
```

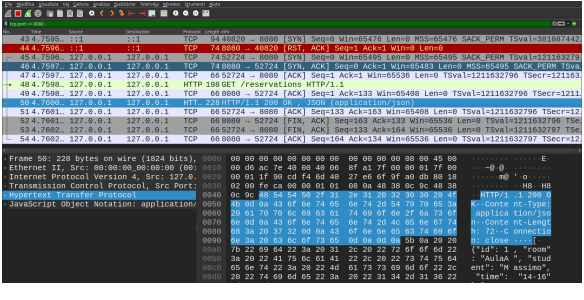
9. Specifiche API REST (CRUD)

Metodo	Endpoint	Azione Interna	Status Code
`GET`	`/reservations`	Serializzazione e lettura in sola lettura	`200 OK`
`POST`	`/reservations`	Validazione Body URL-Encoded, Aggiunta Struct	`201 Created`
`PUT`	`/reservations/{id}`	Ricerca indice array e sovrascrittura valori	`200 OK`
`DELETE`	`/reservations/{id}`	Rimozione elemento e traslazione a sinistra (shift)	`204 No Content`

Evidenza: Autenticazione e JSON



POST Request e Basic Auth



Risposta 200 OK (JSON)

10. Ottimizzazioni Architetturali

Sincronizzazione Centralizzata (Mutex)

L'architettura previene attivamente *race conditions* e corruzione dei dati in memoria. L'impiego rigoroso del **Mutex** (`pthread_mutex_t`) garantisce che l'accesso alla memoria condivisa avvenga in totale mutua esclusione per ogni Worker Thread, assicurando l'assoluta coerenza transazionale sia in lettura che in scrittura.

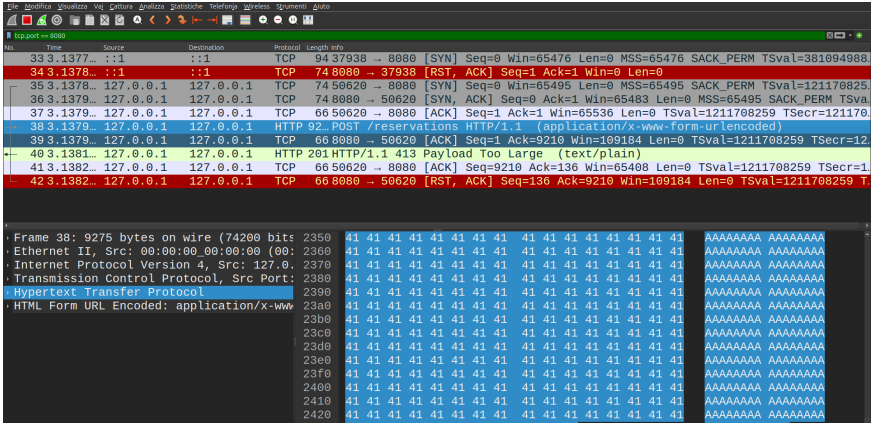
De-frammentazione della Memoria Dinamica

Poiché il C è sprovvisto di Garbage Collector, la rimozione logica di una prenotazione (`DELETE`) espone al rischio di frammentazione del vettore. L'implementazione impiega uno **shift a sinistra** progressivo tramite un **ciclo iterativo for** all'interno dell'area protetta, garantendo la ricompattazione continua dell'array dinamico senza librerie esterne.

11. Sicurezza e Gestione Vulnerabilità

Mitigazione delle criticità intrinseche in ambiente C/POSIX:

- **Socket Leaks:** Dismissione sicura `close(fd)`.
- **Buffer Overflows:** Varianti sicure, errore 413.
- **Interruzione SIGPIPE:** Flag `MSG_NOSIGNAL`.



12. Analisi e Load Testing

Il progetto è stato collaudato sfruttando lo strumento standard **Apache Bench (ab)**. La simulazione ha previsto: **10.000 richieste totali** con un livello di concorrenza di **100 client paralleli**.

Metriche di Prestazione

- **Total Requests:** 10.000
- **Concorrenza Esterna:** 100
- **Success Rate:** 100% (0 Errori)
- **Throughput:** $\sim$ 33.229 Req/Sec

Interpretazione Architetturale

L'assenza totale di connessioni droppate dimostra che la coda semaforica regge il carico e previene i fallimenti TCP. Inoltre, il tempo di risposta mediano di appena **3.0 ms** per richiesta conferma la leggerezza dei thread worker e la corretta implementazione dei lock Mutex sulle risorse condivise.

13. Conclusioni e Sviluppi Futuri

L'implementazione in linguaggio C di un Thread Pool associato a socket BSD si è dimostrata una soluzione *memory-safe* ad altissime prestazioni per la costruzione di architetture HTTP concorrenti.

Sviluppi per Ambienti di Produzione:

1. **Persistenza dei Dati**: Passaggio dalla struttura in RAM ad un database relazionale leggero (come `SQLite3`) per mantenere lo stato dopo il riavvio del processo.
2. **I/O Asincrono (`epoll`)**: Sostituzione delle chiamate bloccanti `read()` con un loop *Event-Driven* nativo Linux per supportare il limite dei 10.000 Socket attivi.
3. **Cifratura**: Implementazione del protocollo *TLS/SSL* tramite librerie come *OpenSSL* per esporre API sicure (HTTPS).

Grazie per l'attenzione.